

CSCN8020 Full Presentation Transcript: Slides 1 to 10

Course: CSCN8020 - Reinforcement Learning | **Project:** Namaste-G1 Humanoid Gesture Learning
Group 4: Ali Cihan Ozdemir, Lohith Reddy Danda, Sumanth Reddy Konannagari, Muthuraj Jayakumar
Total Slides: 10 | **Total Target Duration:** 4 Minutes 30 Seconds

■ **Recommended Presenter Division (4 Group Members):**

Presenter	Slides	Topic / Focus	Time
Presenter 1	Slides 1 & 2	Introduction, Project Overview & Roadmap	~45 sec
Presenter 2	Slides 3 & 4	DQN Baseline Results & Transition to Actor-Critic	~1 min
Presenter 3 (Ali)	Slides 5, 6 & 7	Headline Finding, Zero-Gradient Bug & Post-Fix Results	~1 min 15s
Presenter 4	Slides 8, 9 & 10	Compute Metrics, Mathematical Comparison & Live Demo	~1 min 30s

■ ■ **SLIDE 1: Title & Project Overview**

■ ■ **Target:** 25s | ■ **Word Count:** ~60 words | ■ ■ **Action:** Point to title & team names

Spoken Transcript:

Hello Professor Espinosa and classmates. We are Group 4—Ali Cihan Ozdemir, Lohith Reddy Danda, Sumanth Reddy Konannagari, and Muthuraj Jayakumar.

Today, we are presenting **Namaste-G1: Teaching a Humanoid Robot to Greet Through Reinforcement Learning**.

Our goal is to train a simulated Unitree G1 humanoid robot to reach a two-arm Namaste pose, hold it steadily, and return smoothly to rest.

■ ■ **SLIDE 2: Today’s Plan (Agenda)**

■ ■ **Target:** 20s | ■ **Word Count:** ~50 words | ■ ■ **Action:** Point to roadmap items

Spoken Transcript:

Here is our presentation roadmap for today.

We will review our initial 14-joint DQN baseline, explain why we moved to Actor-Critic, share our major PyTorch gradient bug discovery, analyze post-fix learning stability, explain the math in simple terms, run a live demo, and outline our next steps.

■ ■ **SLIDE 3: Before Actor-Critic: The DQN Baseline**

■ ■ **Target:** 30s | ■ **Word Count:** ~75 words | ■ ■ **Action:** Point to 14-joint stats (3.1 to 5.2 joints)

Spoken Transcript:

To set our baseline, we first built a 14-joint branching Dueling Q-Network with three discrete actions per joint: hold, decrease, or increase.

In a 200-episode diagnostic run, the average number of joints in tolerance improved from 3.1 up to 5.2, reaching a peak moment of 7 out of 14 joints.

However, it achieved zero full 14-joint successes, proving that coarse discrete step choices made fine coordination difficult.

■■ SLIDE 4: Why We Moved to Actor-Critic

■■ Target: 30s | ■ Word Count: ~75 words | ■■ Action: Point to DQN Stepped vs Actor-Critic Smooth

Spoken Transcript:

That limitation motivated our shift to Actor-Critic.

While DQN could only move each joint in fixed discrete steps, Actor-Critic outputs continuous actions for each joint, offering finer and smoother control for 14-joint coordination.

However, continuous control expanded the search space significantly, making the task harder rather than easier.

Our refined goal expanded to three phases: reaching the pose, holding it, and returning smoothly to rest.

■■ SLIDE 5: After Actor-Critic: The Headline Finding

■■ Target: 25s | ■ Word Count: ~60 words | ■■ Action: Point to headline quote box

Spoken Transcript:

We built the full Actor-Critic pipeline, running over 7,800 episodes cleanly without crashes or NaN values.

However, early runs produced zero successes.

Our major headline finding was that slow training was not the core issue—the Actor network received zero working gradient updates until a critical bug was discovered and fixed.

■■ SLIDE 6: The Bug: rsample() + log_prob() = Zero Gradient

■■ Target: 35s | ■ Word Count: ~85 words | ■■ Action: Point to zero vs flowing gradient flow

Spoken Transcript:

When inspecting network weights, we found Actor weights at episode 50 and 300 were byte-identical. The Critic updated, but the Actor remained frozen.

The root cause was that calling PyTorch's rsample() together with log_prob() on the exact same action sample produced a mathematical zero gradient.

Once confirmed, we switched to dist.sample() for training actions, which unlocked gradient flow and enabled the Actor to start learning.

■■ SLIDE 7: After the Fix: Real Learning, Still Unstable

■■ Target: 25s | ■ Word Count: ~65 words | ■■ Action: Point to 6 stabilization attempts

Spoken Transcript:

Immediately after fixing the bug, the policy moved for the first time.

We tested six stabilization attempts: lower learning rates, gradient clipping, reward tuning, longer runs, warm-start initialization, and batched updates.

Although joint movement improved, the policy oscillated instead of holding a clean pose, demonstrating that vanilla Actor-Critic remains unstable for 14-joint continuous control.

■■ SLIDE 8: Compute Time & Timeline Breakdown

■■ Target: 30s | ■ Word Count: ~75 words | ■■ Action: Point to 8,250 episodes & 60%+ compute circle

Spoken Transcript:

In total, we ran 8,250 episodes across 5.4 hours of wall-clock training time.

Over 60% of our compute budget was spent before discovering the gradient bug, where the Actor was mathematically unable to learn.

The primary bottleneck was CPU physics simulation in MuJoCo, requiring up to 4,000 steps per episode. We estimate ~5,000 to 10,000 additional episodes are needed for stable convergence.

■■ SLIDE 9: The Math Behind It: DQN vs. Actor-Critic

■■ Target: 35s | ■ Word Count: ~85 words | ■■ Action: Point to DQN TD target vs Actor-Critic Policy Gradient

Spoken Transcript:

Mathematically, the difference between both methods comes down to action representation:

DQN learns one state-action value per discrete pair, updating toward a bootstrapped TD target: $y = r + \gamma \max_{a'} Q(s', a')$.

Actor-Critic uses the Critic to estimate state value $V(s)$, calculating the Advantage $A(s, a) = r + \gamma V(s') - V(s)$ to update policy probabilities directly.

In short: DQN memorizes exact discrete action values, while Actor-Critic sharpens a continuous policy based on outcome surprise.

■■ SLIDE 10: Live Demo Flow

■■ Target: 25s | ■ Word Count: ~60 words | ■■ Action: Point to 3-step demo table

Spoken Transcript:

We will now show our live simulation demo in 3 quick steps:

- 1. First, target_pose.py shows the target Namaste pose.*
- 2. Second, watch.py shows our DQN baseline reaching up to 7 out of 14 joints.*
- 3. Third, watch_ac.py shows our Actor-Critic model moving toward the pose after our gradient bug fix.*

Thank you, and we welcome any questions!